

TLS Crypto Offload to Network Devices

Boris Pismenny

Netdev conference, Tokyo, October 2016

Mellanox TECHNOLOGIES

Connect. Accelerate. Outperform.™

Motivation

- Encrypted communication is becoming ubiquitous
- How CPU intensive is crypto?
- Compared 2 TLS cipher suites AES128-GCM vs. NULL using OpenSSL single stream

Cipher suite (symmetric crypto-digest)	CPU time (user/system/usage)	Bandwidth
AES128-GCM	0.54 sec/0.78 sec/94%	6.48 Gbit/s
NULL	0.08 sec/0.61 sec/81%	10.4 Gbit/s



85% reduced CPU

© 2015 Mellanox Technologies

Existing Solutions: TOE + TLS

- Not as robust and well tested as the software TCP stack running on the host
- Difficult to maintain
- Likely to have inferior congestion control and retransmission polices
- Lags behind state of the art software implementation

© 2015 Mellanox Technologies

Existing Solutions: TOE + TLS

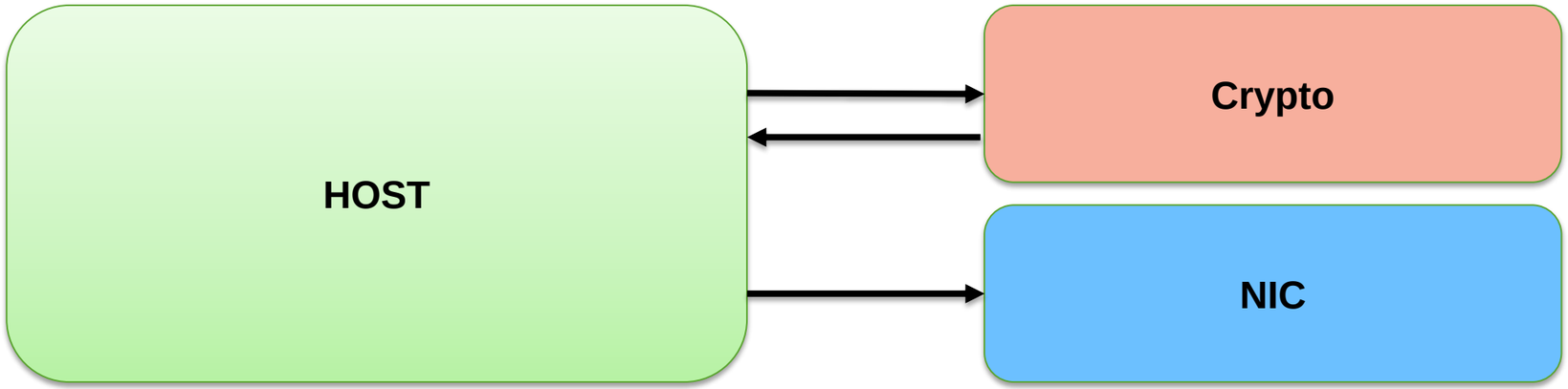
- Not as robust and well tested as the software TCP stack running on the host
- Difficult to maintain
- Likely to have inferior cong control and retransmission polic
- Lags behind state of the lementation



Existing Solutions: Memory To Memory Crypto

- Inefficient for packet processing
  - Multiple PCI crossings
  - Additional processing overhead
    - Prepare crypto descriptors
    - Handle crypto completion interrupt
- Adds significant latency

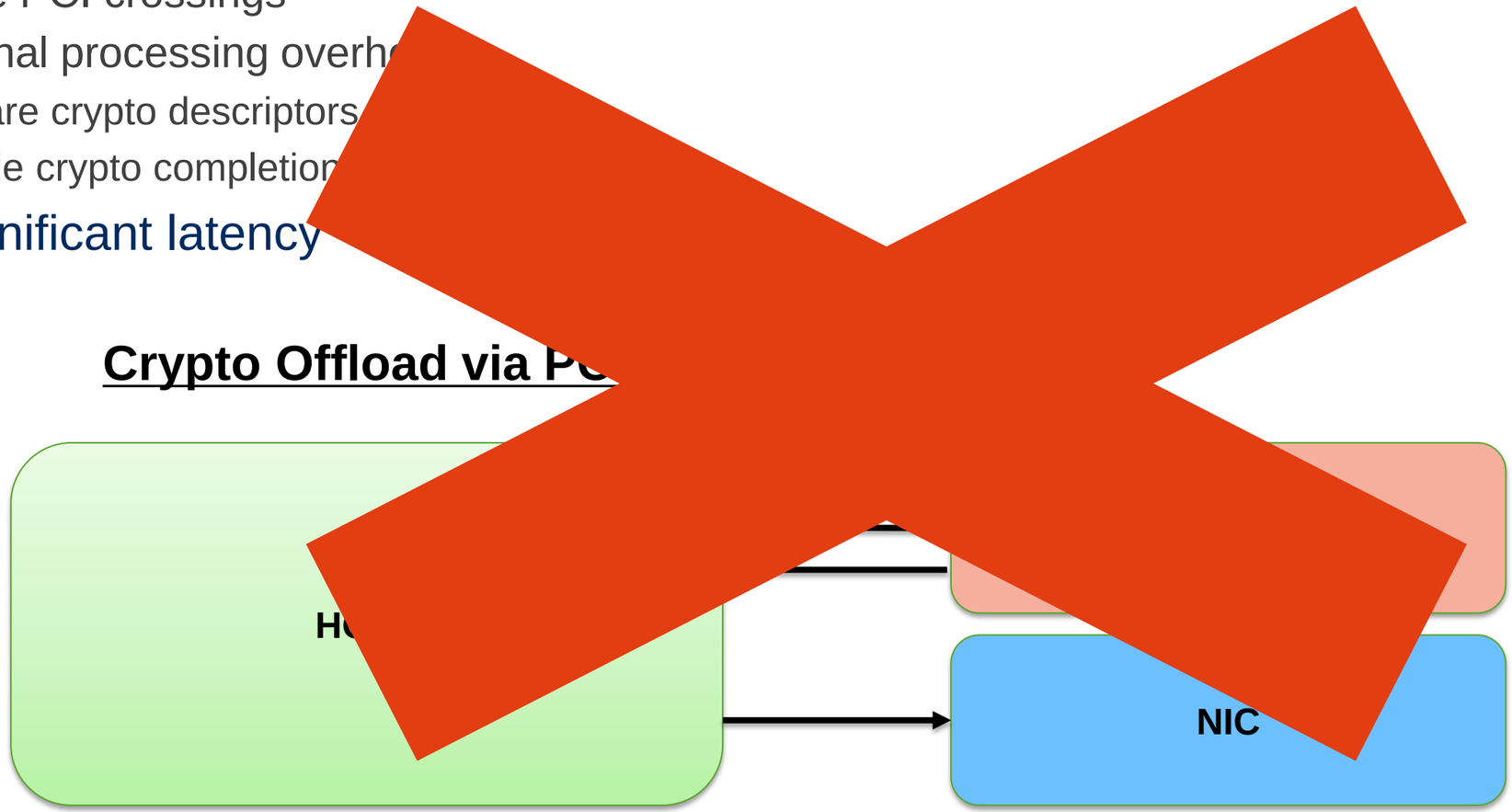
Crypto Offload via PCIe:



© 2015 Mellanox Technologies

Existing Solutions: Memory To Memory Crypto

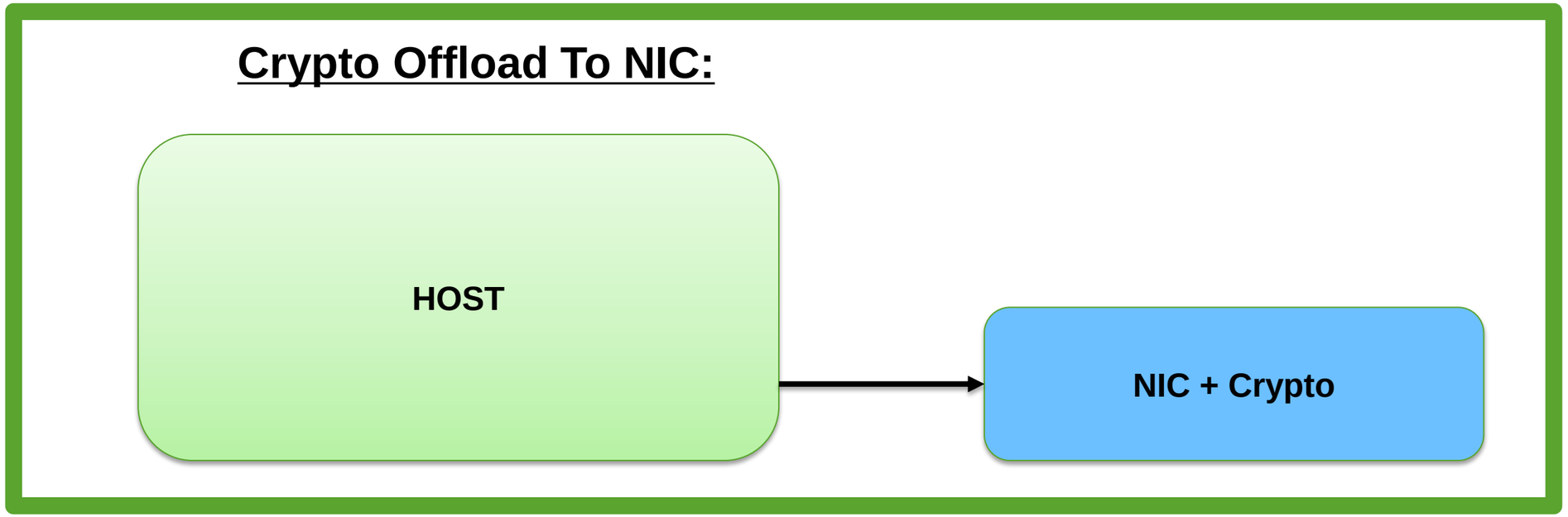
- Inefficient for packet processing
  - Multiple PCI crossings
  - Additional processing overh
    - Prepare crypto descriptors
    - Handle crypto completion
- Adds significant latency



© 2015 Mellanox Technologies

TLS Offload To Network Devices

- NIC holds the crypto state
- NIC encrypts / decrypts packets on the fly
- Cipher text is never held in RAM
- Single PCIe round-trip
- Reduce latency



© 2015 Mellanox Technologies

TLS Record Protocol: Application Data

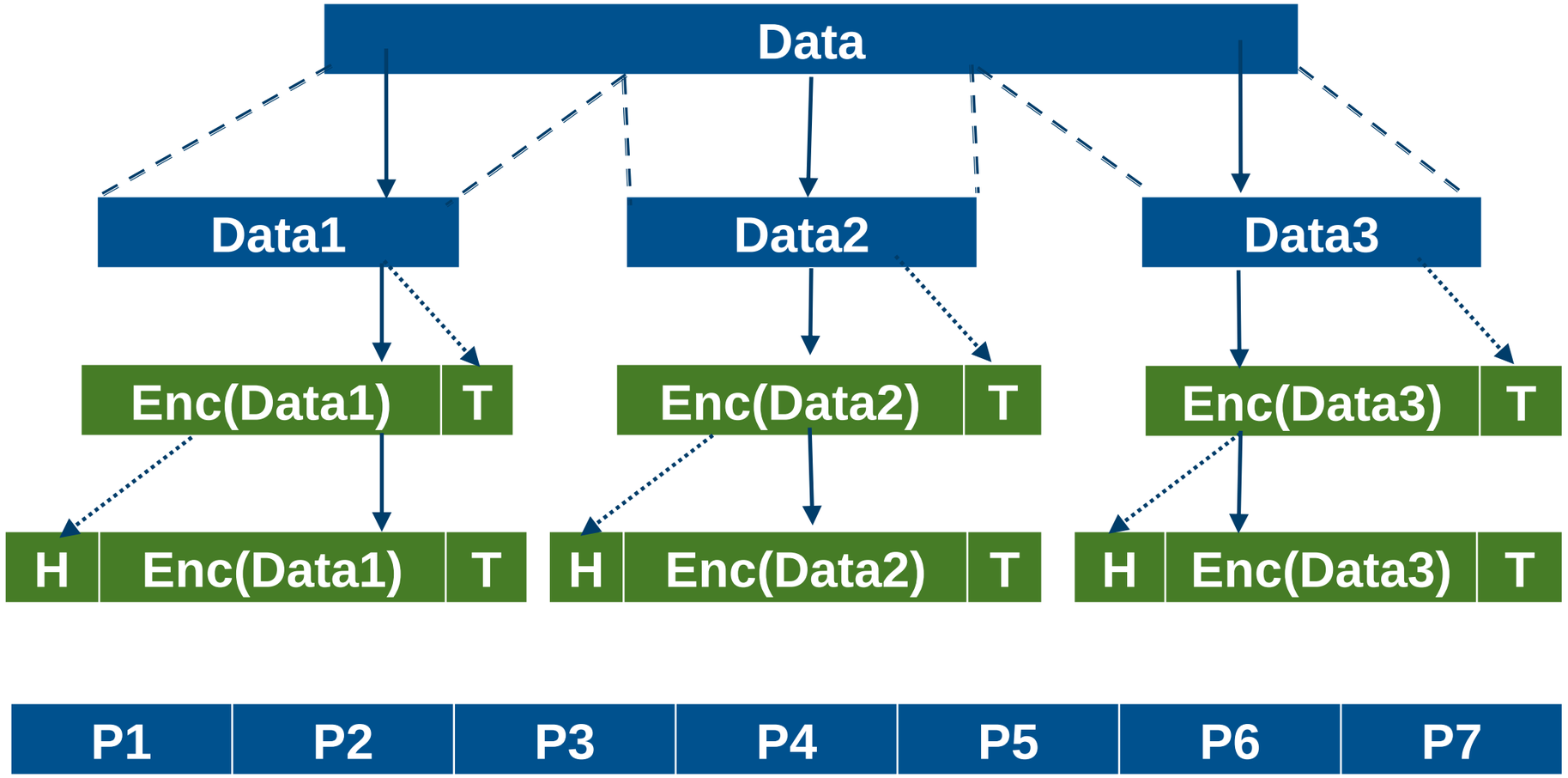
User Space:
Application Data

KTLS:
Fragment (2^14)

KTLS:
Encrypt & Authenticate

KTLS:
TLS Records

TCP:
Segment (MSS)



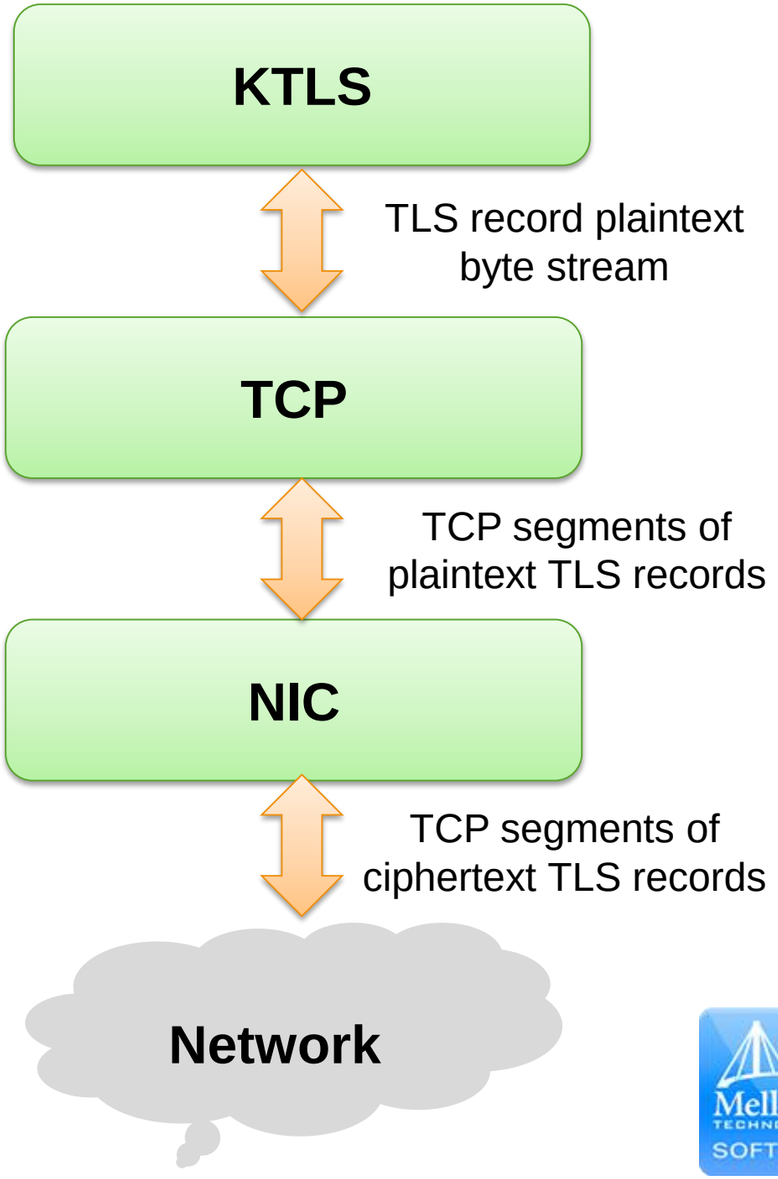
H – TLS Record Header

T – TLS Record Authentication Tag

© 2015 Mellanox Technologies

Software Stack

- Integrates with Kernel TLS
  - Added AF_KTLS “offload” socket option
- Offload symmetric crypto
  - AES-GCM
- Software stack unchanged (even simplified)
  - kTLS (without crypto)
  - TCP/IP
  - Congestion control
  - Memory management



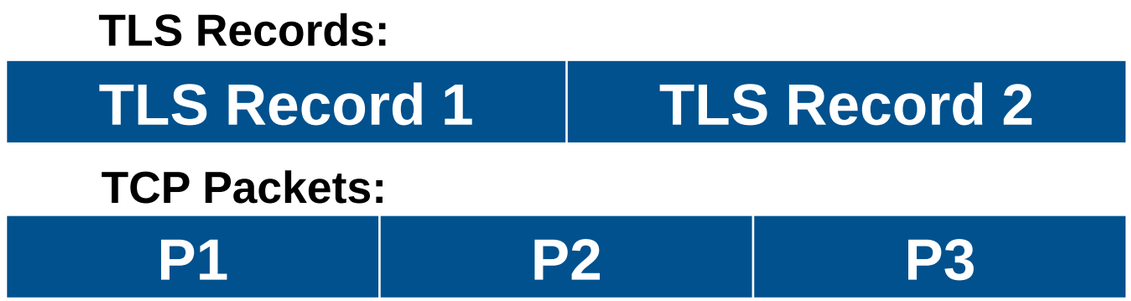
© 2015 Mellanox Technologies

TLS Crypto Offload vs. Other Protocols

- Ideally, packets would be processed independently:
  - IPsec
  - DTLS
  - QUIC

- However ,in TLS **each record** is processed independently.

- Intermediate record state must be tracked by hardware
  - Used by subsequent packets that are part of a previous record



© 2015 Mellanox Technologies

Driver Fast Path

1) Check packet belongs to offloaded socket skb->sk->sk_offloaded - OK
2) Check packet TCP sequence number against expected TCP sequence number - OK
3) Encrypt and authenticate packet in hardware

TLS Records:



TCP Packets:

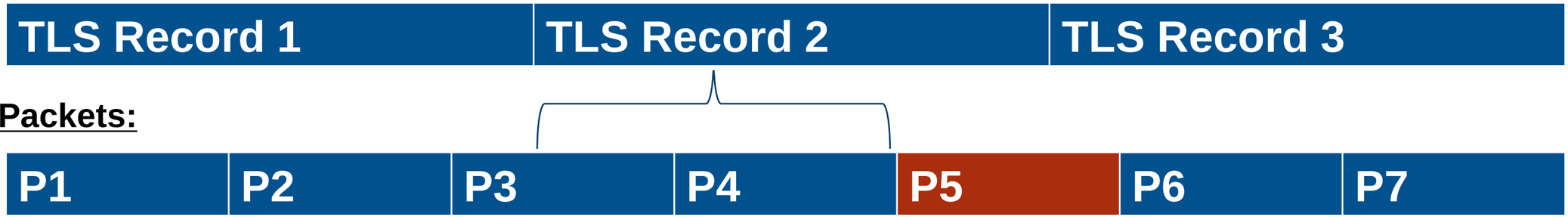


© 2015 Mellanox Technologies

Driver Slow Path – Resync Flow

1) Check packet belongs to offloaded socket skb->sk->sk_offloaded
2) Check packet TCP sequence number against expected TCP sequence number – **Wrong!**
    2.1) **Resync:** Fix hardware TLS context
3) Encrypt and authenticate packet in hardware

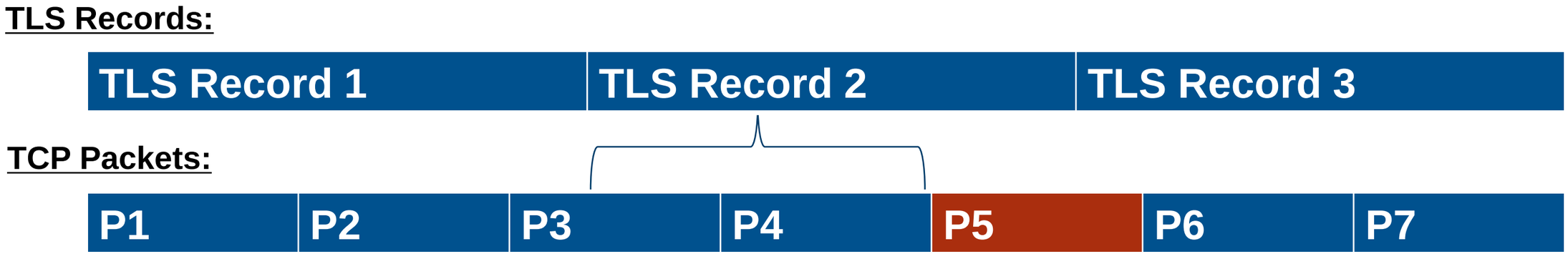
TLS Records:



© 2015 Mellanox Technologies

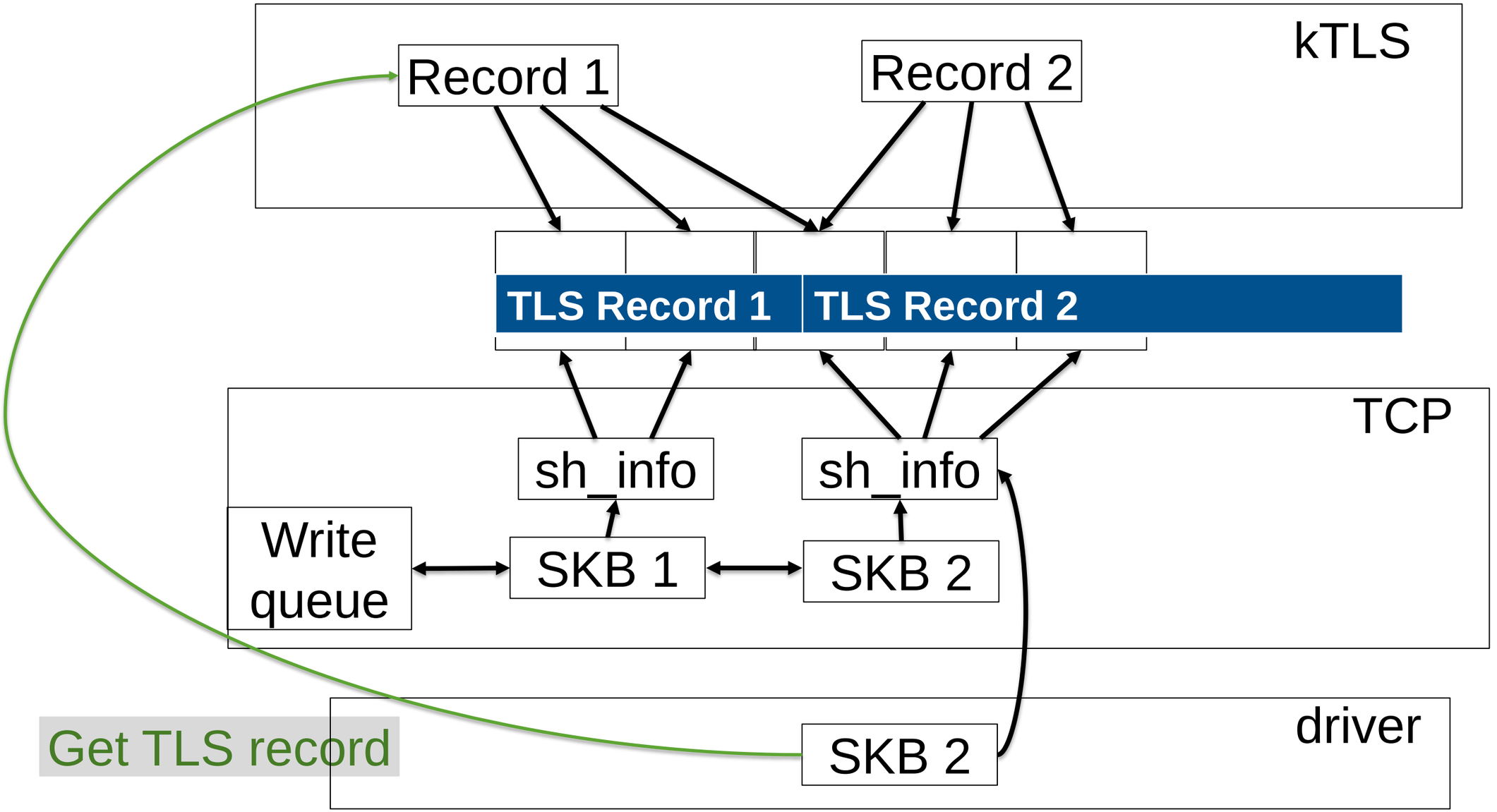
Resync Flow Requirements

- Software needs to keep track of TLS records
- Mapping from SKB to its first TLS record
- TLS records may be released only after the last SKB that overlaps it is ACKed
  - Prevent releasing of partially acknowledged TLS record



© 2015 Mellanox Technologies

Resync Flow


kTLS
Record 1
Record 2
TLS Record 1
TLS Record 2
TCP
sh_info
sh_info
Write queue
SKB 1
SKB 2
driver
SKB 2
Get TLS record

Proposed Solution

kTLS:

- Holds a reference on TLS record data
- Holds a mapping from TCP sequence number to TLS record data
- Expose mapping to offloading device drivers

TCP:

- Call kTLS on tcp_clean_rtx_queue() to release acknowledged TLS records

Driver:

- On Resync query kTLS mapping according to TCP sequence number

© 2015 Mellanox Technologies

Preliminary Performance Results

- TLS single stream AES128-GCM using raw recv (no decrypt) with small TLS records (1457B)

Metric	TCP	GNU TLS	kTLS*	kTLS + Offload
Throughput	10-23Gbps	5.1Gbps	4.3Gbps	8.8Gbps
CPU	100%	100%	100%	100%

*kTLS version from 3 months ago

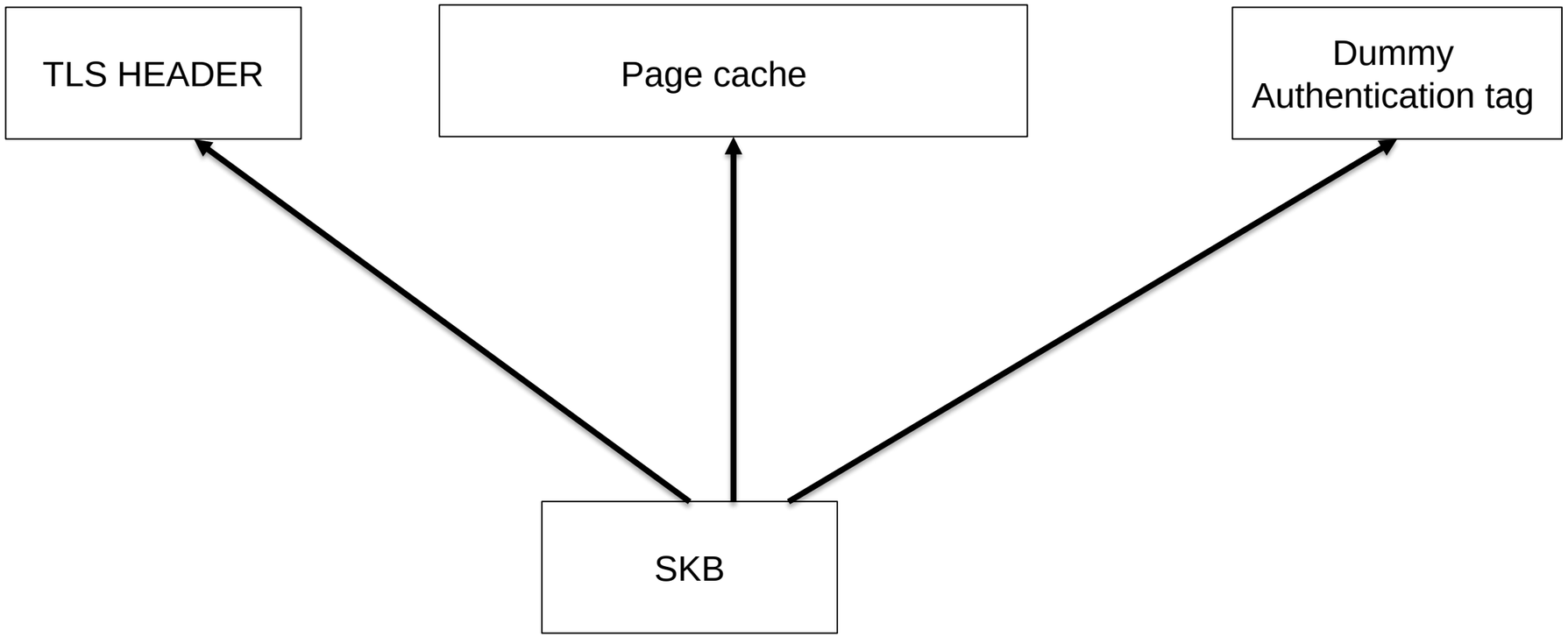
© 2015 Mellanox Technologies

Discussion Items

- Routing changes - plaintext packets must be routed to the offloading device
  - Bind to device
  - Software fallback
- New protocol family AF_KTLS_OFFLOAD
  - In addition to AF_KTLS or another socket option?
- Original TCP socket use
  - Requires flushing kTLS socket
- Memory accounting
  - TLS record accounting
- Zero copy sendfile + crypto offload
  - Retransmission + Stale data

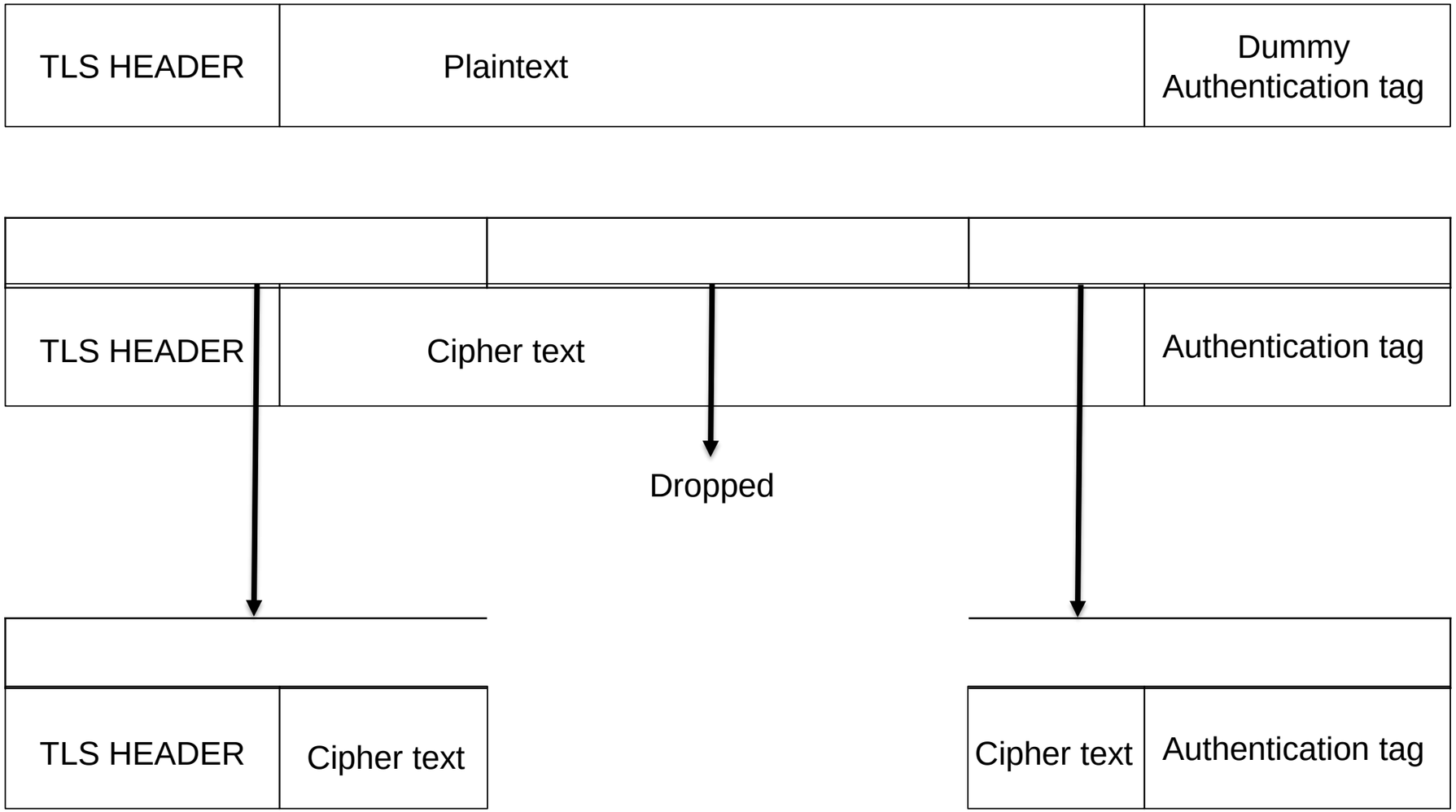
© 2015 Mellanox Technologies

Zero copy sendfile?



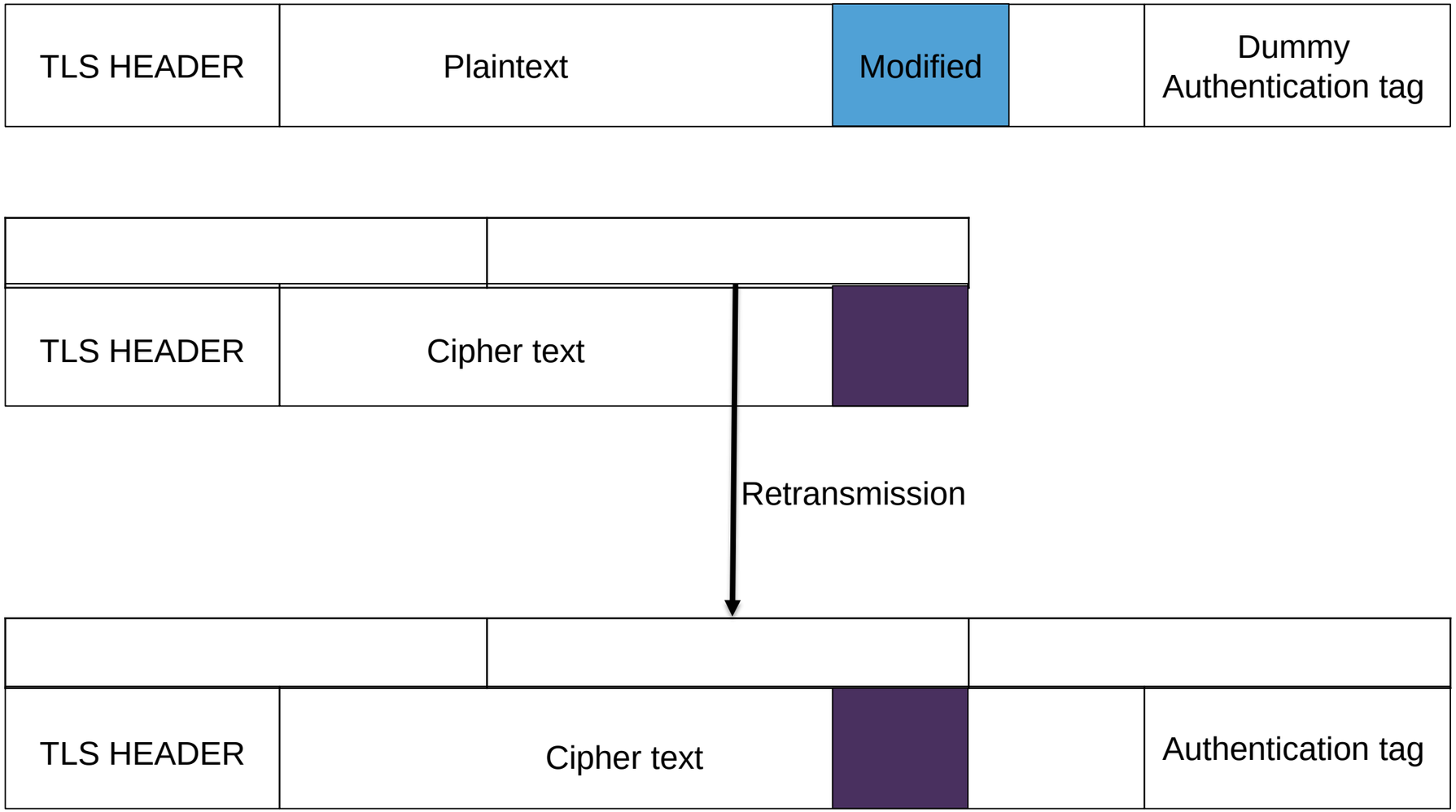
© 2015 Mellanox Technologies

Zero copy sendfile?



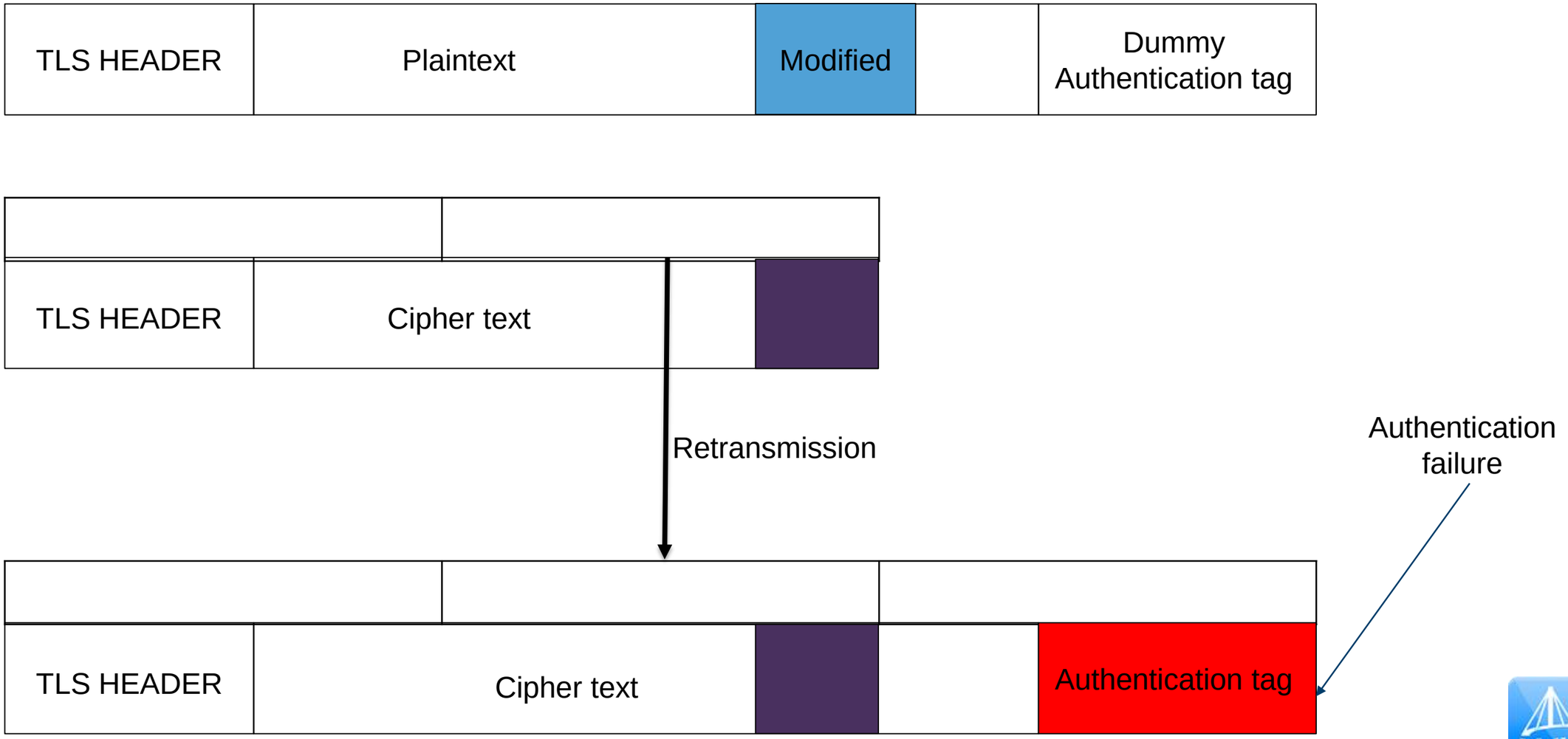
© 2015 Mellanox Technologies

Zero copy sendfile?



© 2015 Mellanox Technologies

Zero copy sendfile?



© 2015 Mellanox Technologies



Thank You

Mellanox TECHNOLOGIES

Connect. Accelerate. Outperform.™